MULTITHREADING NOTES

Definition:

Multithreading is a technique in which multiple threads (small tasks) are executed simultaneously within a single process to improve performance and efficiency.

Need of Multithreading (Real-Time):

1. To perform multiple tasks at the same time.

2. To improve application performance.

3. To keep applications responsive (UI should not freeze).

4. Used in real-world applications like mobile apps, games, browsers, etc.

Basic Syntax:

```
import threading

def task():
    print("Task running")

t = threading.Thread(target=task)
t.start()
t.join()
```

Important Methods:

Thread() → Used to create a thread.

start() → Starts execution of the thread.

join() → Waits for thread to complete.

Example 1: Simple Example

```
import threading

def task1():
    print("Task 1 running")

def task2():
    print("Task 2 running")

t1 = threading.Thread(target=task1)
t2 = threading.Thread(target=task2)

t1.start()
t2.start()

t1.join()
t2.join()
```

Output:

Task 1 running

Task 2 running

Note: Order may change.

Example 2: Downloading, Resume Building, Listening Music

```
import threading
import time

def download():
    for i in range(3):
        print("Downloading...", i)
        time.sleep(1)

def build_resume():
    for i in range(3):
        print("Building resume...", i)
        time.sleep(1)

def listen_music():
    for i in range(3):
        print("Playing music...", i)
        time.sleep(1)

t1 = threading.Thread(target=download)
t2 = threading.Thread(target=build_resume)
t3 = threading.Thread(target=listen_music)

t1.start()
t2.start()
t3.start()

t1.join()
t2.join()
t3.join()

print("All tasks completed")
```

Sample Output:

Downloading... 0 Building resume... 0 Playing music... 0 Downloading... 1 Playing music... 1 Building resume... 1 ... (order may vary) All tasks completed